



JAVA MASTERCLASS

PROGRAMMING ROADMAP

2-Month Beginner to Interview-Ready

PHASE 1 Logic
Foundation

PHASE 2 Algorithms

PHASE 3 DSA Mastery

PHASE 4 Interview Prep

A complete, structured roadmap that teaches you HOW to think — not just WHAT to code

Java Only • Step-by-Step • Logic-First • FAANG-Focused • 350+ Practice Problems

For students who know basic syntax but struggle to think like programmers

TABLE OF CONTENTS

HOW PROGRAMMERS THINK	<i>A Framework for Problem Solving</i>	Week 1
PHASE 1 — Java & Logic Foundation	<i>Week 1–2 Logic, Loops, Arrays, Strings</i>	Week 1-2
60+ Beginner Coding Questions	<i>Printing → Patterns → Primes → Arrays</i>	Week 1-2
PHASE 2 — Core Algorithms	<i>Searching, Sorting, 11 Patterns</i>	Week 3-4
Linear & Binary Search	<i>Find elements efficiently</i>	Week 3
Bubble, Selection, Insertion Sort	<i>Classic sorting algorithms</i>	Week 3
Merge Sort & Quick Sort	<i>Divide and conquer</i>	Week 3
Two Pointer & Sliding Window	<i>Array optimization patterns</i>	Week 4
Prefix Sum, Hashing, Frequency Counter	<i>Counting & lookup patterns</i>	Week 4
Recursion & Backtracking Intro	<i>Thinking recursively</i>	Week 4
Greedy & Bit Manipulation	<i>Greedy choices & binary tricks</i>	Week 4
PHASE 3 — DSA Foundation	<i>Arrays → Graph + Problem Roadmaps</i>	Week 5-6
Arrays & Strings DSA	<i>Core data structures</i>	Week 5
Linked List, Stack, Queue	<i>Linear structures</i>	Week 5
HashMap, HashSet, Trees, BST	<i>Non-linear structures</i>	Week 6
Heaps & Graph Basics	<i>Advanced structures</i>	Week 6
PHASE 4 — Interview Preparation	<i>Daily Routine, Mock Tests, Strategy</i>	Week 7-8
BONUS — Resources & Habits	<i>Books, Sites, YouTube, Daily Habits</i>	—

■ HOW PROGRAMMERS THINK

The Programmer's Mental Framework

Before writing a single line of code, top programmers follow a disciplined thinking process. This framework is the secret weapon that separates strong coders from those who get stuck. Master this process first — the code will follow naturally.

STEP 1: UNDERSTAND THE PROBLEM	Read the problem 3 times. Ask: What is given? What is expected? What are the constraints? What happens at edge cases (empty input, single element, very large numbers)? Never start coding until you can explain the problem in your own words.
STEP 2: THINK IN EXAMPLES	Create 3–4 manual examples. Trace through them by hand. This reveals the hidden pattern. Example: 'Find the largest number in an array' → try [3,1,4,1,5] and trace how you'd find 5 manually.
STEP 3: IDENTIFY THE PATTERN	Ask yourself: Have I seen a similar problem before? Is this a search problem? A comparison problem? A transformation problem? Pattern recognition comes from solving hundreds of problems — this is why volume matters.
STEP 4: BRUTE FORCE FIRST	Always start with the simplest, most obvious solution — even if it's slow. A working brute force solution is better than no solution. It proves you understand the problem and gives a baseline to optimize.
STEP 5: OPTIMIZE	Ask: What is repeated? Can I precompute something? Can I use a better data structure? Is there a mathematical insight? Move from $O(n^2)$ to $O(n)$ by recognizing patterns like hashing, two pointers, or sorting.
STEP 6: CODE IT CLEAN	Write clean, readable code with meaningful variable names. Add comments for each logical section. Handle edge cases explicitly. Don't try to be clever — clear code wins.
STEP 7: TEST & DEBUG	Run your code on all your examples. Test edge cases: empty array, single element, all same elements, negative numbers, maximum size. Debug by adding print statements to trace values.

How to Do a Dry Run

A DRY RUN means manually tracing your code step-by-step on paper before running it on a computer. This is the single most powerful debugging and understanding tool available to you.

Step	Action	Example (Find max in [3,1,4,1,5])
1	Initialize variables	<code>max = arr[0] = 3, i = 1</code>

2	Loop iteration i=1	arr[1]=1, 1 < 3 → max stays 3
3	Loop iteration i=2	arr[2]=4, 4 > 3 → max = 4
4	Loop iteration i=3	arr[3]=1, 1 < 4 → max stays 4
5	Loop iteration i=4	arr[4]=5, 5 > 4 → max = 5
6	Loop ends	Return max = 5 ✓

■ **RULE: Dry run EVERY program you write for the first month. This builds the habit of thinking before coding.**

■ PHASE 1 — JAVA & LOGIC FOUNDATION | WEEK 1–2

Week 1 — Java Foundations

This phase is NOT about memorizing syntax. It's about training your brain to think step-by-step. Every concept learned here will be used in every DSA problem you ever solve.

◆ Variables & Data Types	int, long, double, char, boolean, String. Know their sizes and when to use each.
◆ Operators	Arithmetic (+,-,*,/,%), Relational (==,!=,<,>), Logical (&& ,!), Assignment (=,+=,-=). Master operator precedence.
◆ Conditions	if, else if, else, switch. Think: 'What decision is the program making here?'
◆ Loops	for, while, do-while. Understand: initialization → condition → body → update. Loops are the backbone of all algorithms.
◆ Nested Loops	Loop inside a loop. Critical for matrix problems, pattern printing, bubble sort. Trace both loops in your dry run.
◆ Functions/Methods	Input → Process → Output. Learn return types, parameters, recursion basics. Functions break big problems into small ones.
◆ Arrays Basics	int[] arr = new int[5]. Index starts at 0. Traversal, insertion, finding max/min. The most important data structure.
◆ Strings Basics	charAt(), length(), substring(), contains(), equals(). Strings are character arrays — think of them that way.

60+ Beginner Coding Questions — Week 1 & 2

Solve these IN ORDER. Each problem builds on the last. Do NOT skip any. The goal is not speed — the goal is understanding WHY the solution works.

#	Problem	Topic	Difficulty	Logic / Why It Matters
1	Print Hello World	Printing	Easy	Learn output syntax. Every program starts with printing.

#	Problem	Topic	Difficulty	Logic / Why It Matters
2	Print numbers 1 to 100	Printing/Loops	Easy	Basic for loop. Understand the counter variable.
3	Print even numbers 1–50	Loops/Conditions	Easy	Combine loop with <code>if(n%2==0)</code> . Modulo operator mastery.
4	Print multiplication table of N	Loops	Easy	Nested multiplication logic. Input handling practice.
5	Print sum of 1 to N	Loops/Math	Easy	Accumulator pattern — used in 80% of loop problems.
6	Check if number is positive, negative or zero	If-Else	Easy	3-way branching. Learn if-else if-else chain.
7	Find largest of two numbers	If-Else	Easy	Simple comparison. Foundation of sorting algorithms.
8	Find largest of three numbers	If-Else	Easy	3-variable comparison. Forces logical thinking.
9	Check if number is even or odd	If-Else/Math	Easy	Modulo trick: <code>n%2==0</code> . Used everywhere.
10	Simple calculator (+,-,*,/)	If-Else/Switch	Easy	Learn switch statement. Understand division by zero.
11	Count digits in a number	Loops/Math	Easy	Divide by 10 until 0. Loop termination logic.
12	Reverse a number	Loops/Math	Easy	Extract last digit: <code>n%10</code> . Reverse logic building.
13	Sum of digits of a number	Loops/Math	Easy	Extract & accumulate digits. Very common pattern.
14	Print digits of a number in reverse	Loops/Math	Easy	Combine digit extraction with printing.
15	Count vowels in a string	Loops/Strings	Easy	Loop through chars. Teach <code>charAt()</code> usage.
16	Check if number is palindrome	Loops/Math	Easy	Reverse the number and compare. Classic logic test.
17	Check if string is palindrome	Strings/Two-Pointer	Easy	Compare first and last. Preview of two-pointer pattern.

#	Problem	Topic	Difficulty	Logic / Why It Matters
18	Find factorial of N	Loops/Recursion	Easy	Multiply 1 to N. Understand factorial definition deeply.
19	Print Fibonacci series up to N	Loops	Easy	Two-variable rolling sum. Critical for recursion later.
20	Check if number is prime	Loops/Math	Medium	Check divisibility up to $\sqrt{n}$. Learn sqrt optimization.
21	Print all primes up to N (Sieve intro)	Loops/Math	Medium	Nested loop approach first. Then Sieve concept.
22	Find GCD of two numbers	Math/Algorithms	Medium	Euclidean algorithm. Most elegant 3-line algorithm.
23	Find LCM of two numbers	Math	Easy	$LCM = (a*b)/GCD$. Builds on GCD knowledge.
24	Check if number is Armstrong	Loops/Math	Medium	Sum of cubes of digits. Digit extraction + power.
25	Print all Armstrong numbers 1-999	Loops/Math	Medium	Combine Armstrong check with loop. Good practice.
26	Print right-angle star triangle (rows=N)	Nested Loops	Easy	Outer=rows, inner=cols. Pattern fundamental.
27	Print inverted right triangle	Nested Loops	Easy	Reverse the inner loop count. Think backwards.
28	Print number triangle	Nested Loops	Easy	Print row number instead of star. Minor variation.
29	Print pyramid pattern	Nested Loops	Medium	Spaces + stars formula. Learn centering logic.
30	Print diamond pattern	Nested Loops	Medium	Upper + lower pyramid. Combine two loops.
31	Print hollow square	Nested Loops	Medium	Print border only. $if(i==0 i==n-1 j==0 j==n-1)$.
32	Print Pascal's Triangle	Nested Loops/Math	Medium	$C(n,r)$ formula or accumulation. Classic challenge.

#	Problem	Topic	Difficulty	Logic / Why It Matters
33	Print Floyd's Triangle	Nested Loops	Easy	Simple counter across rows. Auto-increment numbers.
34	Print spiral number matrix	Arrays/Logic	Hard	4-direction traversal. Very common interview warm-up.
35	Find max element in array	Arrays	Easy	Linear scan with max variable. O(n) traversal.
36	Find min element in array	Arrays	Easy	Same as max but with min. Practice variable tracking.
37	Find sum and average of array	Arrays	Easy	Accumulator pattern on arrays. Foundation.
38	Reverse an array	Arrays/Two-Pointer	Easy	Swap first and last, move inward. Preview Two-Pointer.
39	Check if array is sorted	Arrays	Easy	Compare adjacent elements. Returns false on violation.
40	Count occurrences of element X	Arrays	Easy	Linear search + counter. Frequency concept.
41	Remove duplicates from sorted array	Arrays	Medium	Two-pointer approach. Keep only unique elements.
42	Rotate array by K positions	Arrays	Medium	Reverse-based rotation. Very common interview question.
43	Find second largest element	Arrays	Medium	Track top-2 in single pass. Logic exercise.
44	Merge two sorted arrays	Arrays	Medium	Two-pointer merge. Core of Merge Sort.
45	Reverse a string	Strings	Easy	charAt from end to start. StringBuilder for efficiency.
46	Count uppercase and lowercase letters	Strings	Easy	Character comparison. ASCII awareness.
47	Remove all spaces from string	Strings	Easy	Build new string excluding spaces. Filter pattern.
48	Check if two strings are anagrams	Strings/Arrays	Medium	Sort both and compare, OR frequency array.

#	Problem	Topic	Difficulty	Logic / Why It Matters
49	Find first non-repeating character	Strings/HashMap	Medium	Frequency count then scan. HashMap preview.
50	Longest word in a sentence	Strings	Medium	Split by spaces, compare lengths. String parsing.
51	Count words in a string	Strings	Easy	Split and count. Or manual space counting.
52	Replace vowels with *	Strings	Easy	Build new string with conditional replacement.
53	Check if string contains only digits	Strings	Easy	Character.isDigit() method. Input validation.
54	Find all substrings of a string	Strings/Loops	Medium	Nested loop: start to end. $O(n^2)$ pattern.
55	Print numbers 1–N without using loop (recursion)	Recursion	Medium	Base case: $n==0$. Recursive call: print($n-1$) first.
56	Calculate power(base, exp) recursively	Recursion	Medium	$base * power(base, exp-1)$. Fast power preview.
57	Binary to decimal conversion	Math/Loops	Medium	Positional value: $bit * 2^{position}$. Bit manipulation intro.
58	Decimal to binary conversion	Math/Loops	Medium	Divide by 2, collect remainders. Read remainders backwards.
59	Find missing number in 1 to N	Arrays/Math	Medium	Sum formula: $N*(N+1)/2$ minus array sum. Very elegant.
60	Swap two variables without third	Math/Bit	Easy	XOR swap or math swap. Bit manipulation preview.
61	Count set bits (1s) in binary of N	Bit/Math	Medium	$n \& 1$ check + right shift. Core bit manipulation.
62	Check if number is power of 2	Bit Manipulation	Medium	$n \& (n-1) == 0$. Famous bit trick. Very frequent interview Q.
63	Find duplicate number in array of 1-N	Arrays/Math	Medium	Sum approach or XOR approach. Two solutions.

#	Problem	Topic	Difficulty	Logic / Why It Matters
64	Two sum: find if pair sums to target	Arrays/Hashing	Medium	Brute $O(n^2)$ first, then HashMap $O(n)$. Pattern preview.

■ PHASE 2 — CORE ALGORITHMS & PROBLEM-SOLVING PATTERNS | WEEK 3–4

This is the most important phase of your journey. Algorithms are the tools. Problem-solving patterns are the strategies. Master these and you can solve any interview question.

Understanding Time & Space Complexity First

Notation	Name	Example	Performance
$O(1)$	Constant	Array index access	■ Excellent
$O(\log n)$	Logarithmic	Binary Search	■ Excellent
$O(n)$	Linear	Linear Search, single loop	■ Good
$O(n \log n)$	Linearithmic	Merge Sort, Quick Sort avg	■ Good
$O(n^2)$	Quadratic	Bubble Sort, nested loops	■ Avoid for large n
$O(2^n)$	Exponential	Brute force subsets	■ Only for tiny n
$O(n!)$	Factorial	Permutations brute force	■ Never for large n

SECTION A — SEARCHING ALGORITHMS

■ 1. LINEAR SEARCH

WHAT IT SOLVES	Find a target element in an unsorted collection
WHY IT EXISTS	The simplest search — check every element one by one. Works on unsorted data. The baseline against which all other searches are measured.
REAL-WORLD ANALOGY	Checking every seat in a cinema to find your friend — you start from row 1 and go one by one.
INTUITION	Scan element by element. Return index when found, -1 if not found after full scan.

DRY RUN / STEP-BY-STEP THINKING:

Step 1: Start at index 0

Step 2: Compare `arr[i]` with target

Step 3: If match → return i

Step 4: If no match → move to i+1

Step 5: If i reaches end → return -1

JAVA IMPLEMENTATION:

```
public static int linearSearch(int[] arr, int target) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == target) {
            return i; // Found! Return index
        }
    }
    return -1; // Not found
}
```

Time Complexity	Space Complexity
$O(n)$ worst case — must check all n elements	$O(1)$ — no extra space needed

BEGINNER MISTAKES:

- *Forgetting to return -1 for not found*
- *Using == for String comparison (use .equals())*
- *Off-by-one error: $i < arr.length$, NOT $i \leq arr.length$*

INTERVIEW IMPORTANCE: ★★★★★ — Appears in string search, unsorted data, base for optimized searches

EASY PROBLEMS:

- ✓ Find element in array
- ✓ First occurrence of target
- ✓ Check if value exists in list

MEDIUM PROBLEMS:

- ◆ Find all indices where target appears
- ◆ Search in string
- ◆ Count occurrences

WHEN NOT TO USE: When data is sorted → use Binary Search instead ($O(\log n)$ vs $O(n)$)

■ 2. BINARY SEARCH

WHAT IT SOLVES	Find a target element in a SORTED collection in $O(\log n)$ time
WHY IT EXISTS	By exploiting the sorted order, we can eliminate HALF the search space with each comparison. This is the most powerful simple algorithm.
REAL-WORLD ANALOGY	Finding a word in a dictionary: open to the middle, decide left or right half, repeat. You never read every page.

INTUITION

Keep cutting the search space in half. $mid = (low + high) / 2$. If $arr[mid] == target$, found. If $target > arr[mid]$, search right half. If $target < arr[mid]$, search left half.

DRY RUN / STEP-BY-STEP THINKING:

- Step 1: Set $low = 0$, $high = n - 1$
- Step 2: Compute $mid = low + (high - low) / 2$ (avoid overflow!)
- Step 3: If $arr[mid] == target \rightarrow$ return mid
- Step 4: If $arr[mid] < target \rightarrow low = mid + 1$ (search right)
- Step 5: If $arr[mid] > target \rightarrow high = mid - 1$ (search left)
- Step 6: Repeat until $low > high \rightarrow$ return -1

JAVA IMPLEMENTATION:

```
public static int binarySearch(int[] arr, int target) {
    int low = 0, high = arr.length - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2; // Prevents overflow
        if (arr[mid] == target) return mid;
        else if (arr[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}
```

Time Complexity	Space Complexity
$O(\log n)$ — halves search space each step. 1 billion elements $\rightarrow$ 30 comparisons!	$O(1)$ iterative, $O(\log n)$ recursive

BEGINNER MISTAKES:

- Using $mid = (low + high) / 2$ — causes integer overflow for large arrays!
- Forgetting the array **MUST** be sorted first
- Wrong condition: $while(low < high)$ instead of $while(low <= high)$
- Not handling the case where target is at $arr[0]$ or $arr[n-1]$

INTERVIEW IMPORTANCE: ★★★★★ — Appears in 30%+ of all medium LeetCode problems. Must master completely.

EASY PROBLEMS:

- ✓ LeetCode #704 Binary Search
- ✓ LeetCode #35 Search Insert Position
- ✓ Find if element exists

MEDIUM PROBLEMS:

- ◆ LeetCode #34 First and Last Position
- ◆ LeetCode #33 Search in Rotated Sorted Array
- ◆ LeetCode #153 Find Minimum in Rotated Array

WHEN NOT TO USE: When array is unsorted — sort first ($O(n \log n)$) or use linear search

SECTION B — SORTING ALGORITHMS

■ 3. BUBBLE SORT

WHAT IT SOLVES	Sort an array by repeatedly swapping adjacent elements if they are out of order
WHY IT EXISTS	Simple to understand and implement. The heavy elements 'bubble up' to the end, one pass at a time. Learning tool for understanding sorting.
REAL-WORLD ANALOGY	Sorting a queue by height: repeatedly walk down the line swapping adjacent pairs who are out of order, until no swaps needed.
INTUITION	Repeat $n-1$ passes. Each pass: compare adjacent pairs, swap if $\text{left} > \text{right}$. After each pass, the largest unsorted element is at its correct position.

DRY RUN / STEP-BY-STEP THINKING:

- Step 1: Outer loop: $n-1$ passes (i from 0 to $n-2$)
- Step 2: Inner loop: compare $\text{arr}[j]$ and $\text{arr}[j+1]$ (j from 0 to $n-i-2$)
- Step 3: If $\text{arr}[j] > \text{arr}[j+1] \rightarrow$ swap them
- Step 4: Optional: if no swap in a pass $\rightarrow$ array is already sorted (early exit)

JAVA IMPLEMENTATION:

```
public static void bubbleSort(int[] arr) {
    int n = arr.length;
    for (int i = 0; i < n-1; i++) {
        boolean swapped = false;
        for (int j = 0; j < n-i-1; j++) {
            if (arr[j] > arr[j+1]) {
                int temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
                swapped = true;
            }
        }
        if (!swapped) break; // Already sorted
    }
}
```

Time Complexity	Space Complexity
$O(n^2)$ worst/average, $O(n)$ best (already sorted with early exit)	$O(1)$ — in-place sorting

BEGINNER MISTAKES:

- Wrong inner loop bound: should be $n-i-1$, not $n-1$
- Not adding the 'swapped' optimization

■ *Confusing with Selection Sort (bubble swaps adjacent; selection finds min)*

INTERVIEW IMPORTANCE: ★★☆☆ — Rarely used in practice, but tested heavily in interviews to test sorting fundamentals

EASY PROBLEMS:

- ✓ Sort an array of 5 elements manually
- ✓ Count swaps needed to sort
- ✓ Sort in descending order

MEDIUM PROBLEMS:

- ◆ Number of swaps to sort array
- ◆ Is array sorted in k passes?

WHEN NOT TO USE: For real sorting tasks — use `Arrays.sort()` or Merge Sort / Quick Sort

■ 4. SELECTION SORT

WHAT IT SOLVES	Sort by repeatedly finding the minimum element and placing it at the correct position
WHY IT EXISTS	More intuitive than bubble sort. Minimizes swaps (max $n-1$ swaps). Good for when swapping is expensive.
REAL-WORLD ANALOGY	Sorting playing cards: scan all cards, pick the lowest, place it first. Then scan remaining, pick next lowest, place it second.
INTUITION	On each pass, find the minimum element in the unsorted portion and swap it with the first unsorted position.

DRY RUN / STEP-BY-STEP THINKING:

- Step 1: Outer loop: i from 0 to $n-2$ (position to fill)
- Step 2: Find minimum in $\text{arr}[i..n-1]$: set $\text{minIdx} = i$, scan j from $i+1$
- Step 3: If $\text{arr}[j] < \text{arr}[\text{minIdx}] \rightarrow$ update $\text{minIdx} = j$
- Step 4: After scan: swap $\text{arr}[i]$ with $\text{arr}[\text{minIdx}]$
- Step 5: Now $\text{arr}[0..i]$ is sorted

JAVA IMPLEMENTATION:

```
public static void selectionSort(int[] arr) {
    int n = arr.length;
    for (int i = 0; i < n-1; i++) {
        int minIdx = i;
        for (int j = i+1; j < n; j++) {
            if (arr[j] < arr[minIdx]) {
                minIdx = j;
            }
        }
        int temp = arr[i];
        arr[i] = arr[minIdx];
        arr[minIdx] = temp;
    }
}
```

}

Time Complexity	Space Complexity
$O(n^2)$ always — no early exit possible	$O(1)$ — in-place

BEGINNER MISTAKES:

- *Not updating minIdx when a smaller element is found*
- *Starting j from i instead of i+1*
- *Swapping on every comparison instead of after finding minimum*

INTERVIEW IMPORTANCE: ★★☆☆ — Good for understanding in-place sorting. Shows up in interviews for comparison.

EASY PROBLEMS:

- ✓ Sort array in ascending order
- ✓ Sort array in descending order
- ✓ Find kth smallest element approach

MEDIUM PROBLEMS:

- ◆ Sort by multiple criteria
- ◆ Stable vs unstable sorting discussion

WHEN NOT TO USE: For large datasets — Selection Sort is too slow. Use Merge Sort or Quick Sort.

■ **5. INSERTION SORT**

WHAT IT SOLVES	Build sorted array one element at a time by inserting each element in its correct position
WHY IT EXISTS	Efficient for small datasets or nearly sorted data. This is how most people sort playing cards in their hand.
REAL-WORLD ANALOGY	Sorting cards in your hand: hold one card, insert it in the correct position among already-sorted cards in your hand.
INTUITION	Take each element (starting from index 1), compare backwards with sorted portion, shift elements right until finding correct position, insert.

DRY RUN / STEP-BY-STEP THINKING:

- Step 1: Start with first element as 'sorted' ($i=1$)
- Step 2: Take $arr[i]$ as 'key' to insert
- Step 3: $j = i-1$. Shift elements $>$ key one position right
- Step 4: While $j \geq 0$ AND $arr[j] >$ key: $arr[j+1] = arr[j]$; $j--$
- Step 5: Place key at $arr[j+1]$

JAVA IMPLEMENTATION:


```
public static void insertionSort(int[] arr) {
    for (int i = 1; i < arr.length; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}
```

Time Complexity	Space Complexity
$O(n^2)$ worst, $O(n)$ best (nearly sorted). Excellent for small/nearly-sorted arrays.	$O(1)$ — in-place

BEGINNER MISTAKES:

- Forgetting $j \geq 0$ condition (causes `ArrayIndexOutOfBoundsException`)
- Using `arr[j] >= key` instead of `arr[j] > key` (breaks stability)
- Not placing key at `arr[j+1]` (off by one on final placement)

INTERVIEW IMPORTANCE: ★★★★★ — Tested often. Adaptive nature ($O(n)$ for sorted input) is a key interview discussion point.

EASY PROBLEMS:

- ✓ Sort small array
- ✓ Sort nearly sorted array of 1000 elements (why best here?)

MEDIUM PROBLEMS:

- ◆ Tim Sort intro (Insertion + Merge combo used in Java's `Arrays.sort`)
- WHEN NOT TO USE: For large random datasets — use Merge Sort instead*

■ 6. MERGE SORT

WHAT IT SOLVES	Sort by recursively dividing array in half, sorting each half, and merging
WHY IT EXISTS	Divide & Conquer paradigm. Guaranteed $O(n \log n)$ always. Stable sort. Used in Java's TimSort for objects.
REAL-WORLD ANALOGY	Sorting two shuffled stacks of papers: split your papers in half, give half to a friend. Both sort independently, then merge by comparing top cards.
INTUITION	Divide: split array in half recursively until 1-element arrays (already sorted). Conquer: merge two sorted halves by comparing elements one by one.

DRY RUN / STEP-BY-STEP THINKING:

- Step 1: Find $mid = (left + right) / 2$
- Step 2: Recursively `mergeSort(arr, left, mid)`
- Step 3: Recursively `mergeSort(arr, mid+1, right)`
- Step 4: Create temp arrays `L[]` and `R[]`
- Step 5: Merge: compare `L[i]` and `R[j]`, place smaller in `arr[k]`
- Step 6: Copy remaining elements from whichever half is not empty

JAVA IMPLEMENTATION:

```
public static void mergeSort(int[] arr, int left, int right) {
    if (left >= right) return; // Base case
    int mid = left + (right - left) / 2;
    mergeSort(arr, left, mid);
    mergeSort(arr, mid + 1, right);
    merge(arr, left, mid, right);
}

private static void merge(int[] arr, int l, int m, int r) {
    int n1 = m - l + 1, n2 = r - m;
    int[] L = new int[n1], R = new int[n2];
    System.arraycopy(arr, l, L, 0, n1);
    System.arraycopy(arr, m+1, R, 0, n2);
    int i = 0, j = 0, k = l;
    while (i < n1 && j < n2)
        arr[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}
```

Time Complexity	Space Complexity
$O(n \log n)$ ALWAYS — best, worst, and average. Most reliable sorting algorithm.	$O(n)$ — requires extra space for temp arrays

BEGINNER MISTAKES:

- *Forgetting base case: if(left >= right) return*
- *Wrong merge: not handling remaining elements after one array is exhausted*
- *Using mid = (left+right)/2 risks overflow for large indices*

INTERVIEW IMPORTANCE: ★★★★★ — Core interview algorithm. Must know implementation. Inversion count problem uses this.

EASY PROBLEMS:

- ✓ LeetCode #912 Sort an Array
- ✓ Sort linked list (Merge Sort shines here)
- ✓ Count inversions in array

MEDIUM PROBLEMS:

- ◆ LeetCode #315 Count of Smaller Numbers After Self
- ◆ LeetCode #493 Reverse Pairs

WHEN NOT TO USE: When extra memory O(n) is not allowed — use Quick Sort (in-place)

■ 7. QUICK SORT

WHAT IT SOLVES	Sort by partitioning around a pivot: elements smaller go left, larger go right, then recurse
WHY IT EXISTS	In-place, cache-friendly, average $O(n \log n)$. Used in Java's <code>Arrays.sort()</code> for primitives. Fastest in practice for random data.
REAL-WORLD ANALOGY	Sorting a line of students by height: pick one student (pivot), students shorter go left, taller go right. Now sort each group the same way.
INTUITION	Pick a pivot. Partition: put all smaller elements to pivot's left, larger to right. Pivot is now at its final sorted position. Recurse on both sides.

DRY RUN / STEP-BY-STEP THINKING:

- Step 1: Choose pivot (last element is simplest)
- Step 2: $i = \text{left} - 1$ (index of smaller element)
- Step 3: j loop from left to right-1
- Step 4: If $\text{arr}[j] \leq \text{pivot}$: $i++$, swap $\text{arr}[i]$ and $\text{arr}[j]$
- Step 5: After loop: swap $\text{arr}[i+1]$ with pivot
- Step 6: Pivot is at $i+1$. Recurse on $[\text{left}..i]$ and $[i+2..\text{right}]$

JAVA IMPLEMENTATION:

```
public static void quickSort(int[] arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
```

```
private static int partition(int[] arr, int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            int temp = arr[i]; arr[i] = arr[j]; arr[j] = temp;
        }
    }
    int temp = arr[i+1]; arr[i+1] = arr[high]; arr[high] = temp;
    return i + 1;
}
```

Time Complexity	Space Complexity
$O(n \log n)$ average, $O(n^2)$ worst case (sorted array with bad pivot). Use random pivot to avoid worst case.	$O(\log n)$ recursive stack space (in-place partitioning)

BEGINNER MISTAKES:

- *Not handling low >= high base case*
- *Pivot choice: always picking first/last on sorted data $\rightarrow O(n^2)$. Use random pivot.*
- *Wrong partition index returned — off by one*

INTERVIEW IMPORTANCE: ★★★★★ — kth largest element, quick select algorithm, fundamental interview algorithm

EASY PROBLEMS:

- ✓ LeetCode #215 Kth Largest Element in Array (Quick Select)
- ✓ Sort array in-place

MEDIUM PROBLEMS:

- ◆ LeetCode #75 Sort Colors (Dutch National Flag)
- ◆ LeetCode #324 Wiggle Sort II

WHEN NOT TO USE: For linked lists — use Merge Sort (Quick Sort needs random access)

SECTION C — PROBLEM-SOLVING PATTERNS

These patterns are the VOCABULARY of problem solving. Once you recognize them, 80% of interview problems become recognizable within 30 seconds.

■ 8. TWO POINTER ALGORITHM

WHAT IT SOLVES	Solve array/string problems with two indices moving toward (or away from) each other
WHY IT EXISTS	Instead of nested loops $O(n^2)$, use two pointers moving smartly to achieve $O(n)$. Reduces time drastically for many problems.
REAL-WORLD ANALOGY	Two people walking toward each other in a hallway — they find each other in half the time of one person checking every spot.
INTUITION	Use when: sorted array, palindrome check, find pair with sum, remove duplicates. One pointer starts at left, one at right, move based on condition.

DRY RUN / STEP-BY-STEP THINKING:

- Step 1: left = 0, right = n-1
- Step 2: While left < right:
- Step 3: Compute current result using arr[left] and arr[right]
- Step 4: If result matches target → found!
- Step 5: If result too small → left++ (increase sum)
- Step 6: If result too large → right-- (decrease sum)

JAVA IMPLEMENTATION:

```
// Two Sum in sorted array
public static int[] twoSum(int[] arr, int target) {
    int left = 0, right = arr.length - 1;
    while (left < right) {
        int sum = arr[left] + arr[right];
        if (sum == target) return new int[]{left, right};
        else if (sum < target) left++;
        else right--;
    }
    return new int[]{-1, -1};
}
```

Time Complexity	Space Complexity
$O(n)$ — both pointers move at most n total steps combined	$O(1)$ — no extra space

BEGINNER MISTAKES:

- Using while(left <= right) instead of while(left < right) for pair problems
- Not sorting the array first when required

■ *Moving wrong pointer — analyze which direction reduces/increases the value*

INTERVIEW IMPORTANCE: ★★★★★ — Used in 25%+ of array problems. Two Sum, Container With Most Water, palindrome, remove duplicates.

EASY PROBLEMS:

- ✓ LeetCode #167 Two Sum II
- ✓ LeetCode #125 Valid Palindrome
- ✓ LeetCode #344 Reverse String

MEDIUM PROBLEMS:

- ◆ LeetCode #11 Container With Most Water
- ◆ LeetCode #15 3Sum
- ◆ LeetCode #42 Trapping Rain Water

WHEN NOT TO USE: When array is unsorted and sorting would lose information — use HashMap instead

■ 9. SLIDING WINDOW ALGORITHM

WHAT IT SOLVES	Find optimal subarray/substring by maintaining a 'window' that slides across the data
WHY IT EXISTS	Instead of recalculating subarray properties from scratch each time $O(n^2)$, maintain a running calculation as window expands/contracts $O(n)$.
REAL-WORLD ANALOGY	A train window slides along a track — what you see changes one element at a time. You don't recalculate your entire view, just add the new and remove the old.
INTUITION	Fixed window: maintain exact size k , slide by removing left and adding right. Variable window: expand right, shrink left when constraint violated.

DRY RUN / STEP-BY-STEP THINKING:

- Step 1: FIXED: start = 0. Compute first window $[0..k-1]$
- Step 2: Slide: add $arr[i]$, subtract $arr[i-k]$. Update result each step.
- Step 3: VARIABLE: right=0, left=0. Expand right in while(right < n) loop
- Step 4: If window invalid → shrink: remove $arr[left]$, left++
- Step 5: Update answer after each valid window

JAVA IMPLEMENTATION:

```
// Max sum subarray of size k (Fixed Window)
public static int maxSumSubarray(int[] arr, int k) {
    int windowSum = 0, maxSum = 0;
    for (int i = 0; i < k; i++) windowSum += arr[i];
    maxSum = windowSum;
    for (int i = k; i < arr.length; i++) {
        windowSum += arr[i] - arr[i-k];
        maxSum = Math.max(maxSum, windowSum);
    }
}
```

```
return maxSum;
}
```

Time Complexity	Space Complexity
$O(n)$ — each element enters and leaves window at most once	$O(1)$ for fixed window, $O(k)$ for variable window with frequency map

BEGINNER MISTAKES:

- *Recomputing the window from scratch each time (defeats the purpose)*
- *Not handling edge case where $k > \text{array length}$*
- *Variable window: not understanding when to shrink vs expand*

INTERVIEW IMPORTANCE: ★★★★★ — Substring problems, max sum, longest subarray with condition. Core FAANG pattern.

EASY PROBLEMS:

- ✓ LeetCode #643 Maximum Average Subarray
- ✓ LeetCode #1343 Number of Sub-arrays of Size K
- ✓ Max sum subarray of size k

MEDIUM PROBLEMS:

- ◆ LeetCode #3 Longest Substring Without Repeating Characters
- ◆ LeetCode #76 Minimum Window Substring
- ◆ LeetCode #239 Sliding Window Maximum

WHEN NOT TO USE: When order of elements doesn't matter or you need non-contiguous subsets — different pattern needed

■ 10. PREFIX SUM ALGORITHM

WHAT IT SOLVES	Precompute cumulative sums to answer range sum queries in $O(1)$ instead of $O(n)$
WHY IT EXISTS	Computing sum from index L to R repeatedly costs $O(n)$ each time. Precomputing prefix sums enables $O(1)$ range queries after $O(n)$ preprocessing.
REAL-WORLD ANALOGY	Odometer in a car: instead of counting every mile you drove in a segment, you read start odometer and end odometer and subtract.
INTUITION	$\text{prefix}[i] = \text{arr}[0] + \text{arr}[1] + \dots + \text{arr}[i]$. Sum from L to R = $\text{prefix}[R] - \text{prefix}[L-1]$. Precompute once, query instantly.

DRY RUN / STEP-BY-STEP THINKING:

- Step 1: Create `prefix[]` of same size as `arr`
- Step 2: `prefix[0] = arr[0]`
- Step 3: For `i` from 1 to `n-1`: `prefix[i] = prefix[i-1] + arr[i]`

Step 4: Query $\text{sum}(L, R) = \text{prefix}[R] - (L > 0 ? \text{prefix}[L-1] : 0)$

JAVA IMPLEMENTATION:

```
public static int[] buildPrefix(int[] arr) {
    int[] prefix = new int[arr.length];
    prefix[0] = arr[0];
    for (int i = 1; i < arr.length; i++)
        prefix[i] = prefix[i-1] + arr[i];
    return prefix;
}

public static int rangeSum(int[] prefix, int l, int r) {
    return l == 0 ? prefix[r] : prefix[r] - prefix[l-1];
}
```

Time Complexity	Space Complexity
$O(n)$ preprocessing + $O(1)$ per query	$O(n)$ for prefix array

BEGINNER MISTAKES:

- Off-by-one: rangeSum from L to R is $\text{prefix}[R] - \text{prefix}[L-1]$, NOT $\text{prefix}[R] - \text{prefix}[L]$
- Not handling $L=0$ separately ($\text{prefix}[-1]$ doesn't exist)
- Integer overflow for large arrays — use `long[]`

INTERVIEW IMPORTANCE: ★★★★★ — Range sum queries, subarray sum equals K , equilibrium index. Very common pattern.

EASY PROBLEMS:

- ✓ LeetCode #303 Range Sum Query Immutable
- ✓ Find subarray with zero sum
- ✓ Count subarrays with even sum

MEDIUM PROBLEMS:

- ◆ LeetCode #560 Subarray Sum Equals K
- ◆ LeetCode #525 Contiguous Array
- ◆ LeetCode #974 Subarray Sums Divisible by K

WHEN NOT TO USE: When you only need one sum (not multiple queries) — just use a running sum

■ 11. FAST & SLOW POINTER (Floyd's Algorithm)

WHAT IT SOLVES	Detect cycles, find midpoints, and solve linked list problems using two pointers at different speeds
WHY IT EXISTS	The slow pointer moves 1 step, fast pointer moves 2 steps. If there's a cycle, they must eventually meet. If no cycle, fast reaches end.
REAL-WORLD ANALOGY	A circular running track: if two runners run at different speeds, the faster one will eventually lap (meet) the slower one. On a straight track, the faster one just finishes first.
INTUITION	slow = head, fast = head. Each step: slow = slow.next, fast = fast.next.next. If cycle exists, they meet. For midpoint: slow is at middle when fast reaches end.

DRY RUN / STEP-BY-STEP THINKING:

- Step 1: Initialize slow = head, fast = head
- Step 2: Loop while fast != null && fast.next != null
- Step 3: slow = slow.next; fast = fast.next.next
- Step 4: If slow == fast → cycle detected
- Step 5: For midpoint: when fast == null, slow is at middle

JAVA IMPLEMENTATION:

```
// Detect cycle in linked list
public boolean hasCycle(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) return true;
    }
    return false;
}

// Find middle of linked list
public ListNode middleNode(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    return slow; // slow is at middle
}
```

Time Complexity	Space Complexity
$O(n)$ — fast pointer covers all n nodes	$O(1)$ — only two pointers

BEGINNER MISTAKES:

- *Checking slow==fast before first move (they start at same node!)*
- *Not checking fast.next != null (NullPointerException on even-length list)*
- *Using this on arrays instead of linked lists — only applicable to sequential access structures*

INTERVIEW IMPORTANCE: ★★★★★ — Linked list cycle detection is a top-5 linked list interview question worldwide.

EASY PROBLEMS:

- ✓ LeetCode #141 Linked List Cycle
- ✓ LeetCode #876 Middle of Linked List
- ✓ Happy Number (cycle in number sequence)

MEDIUM PROBLEMS:

- ◆ LeetCode #142 Linked List Cycle II (find start of cycle)
- ◆ LeetCode #287 Find Duplicate Number
- ◆ LeetCode #202 Happy Number

WHEN NOT TO USE: For arrays with random access — two pointer or binary search is more appropriate

■ 12. HASHING TECHNIQUE & 13. FREQUENCY COUNTER

WHAT IT SOLVES	Use HashMap/HashSet for O(1) lookups instead of O(n) array scans
WHY IT EXISTS	Arrays require O(n) to find an element. HashMaps find any element in O(1) average time by storing key-value pairs. This turns many O(n ²) solutions to O(n).
REAL-WORLD ANALOGY	A phonebook: instead of reading every name to find a number, you jump directly to the right page. HashMap is your phonebook.
INTUITION	Store computed values in a HashMap. Before computing, check if already computed. Frequency Counter: count occurrences of each element, then use those counts.

DRY RUN / STEP-BY-STEP THINKING:

- Step 1: Create HashMap
- Step 2: For each element: check if map.containsKey(target - num) [Two Sum example]
- Step 3: If yes → pair found
- Step 4: If no → map.put(num, index)
- Step 5: Frequency Counter: map.put(num, map.getDefault(num, 0) + 1)

JAVA IMPLEMENTATION:

```
// Two Sum using HashMap - O(n)
public int[] twoSum(int[] nums, int target) {
    Map map = new HashMap<>();
    for (int i = 0; i < nums.length; i++) {
        int complement = target - nums[i];
        if (map.containsKey(complement))
            return new int[]{map.get(complement), i};
    }
}
```

```

        map.put(nums[i], i);
    }
    return new int[]{};
}

// Frequency Counter
public Map frequency(int[] arr) {
    Map freq = new HashMap<>();
    for (int num : arr)
        freq.put(num, freq.getDefault(num, 0) + 1);
    return freq;
}

```

Time Complexity	Space Complexity
O(n) average for all HashMap operations	O(n) for storing n elements

BEGINNER MISTAKES:

- *Using == for key comparison (use .equals() or .containsKey())*
- *Forgetting that HashMap has no guaranteed order (use LinkedHashMap if order matters)*
- *Hash collisions: rare but possible, understand it conceptually*

INTERVIEW IMPORTANCE: ★★★★★ — Two Sum, anagram check, subarray sum, first non-repeat. Appears in 40%+ of problems.

EASY PROBLEMS:

- ✓ LeetCode #1 Two Sum
- ✓ LeetCode #242 Valid Anagram
- ✓ LeetCode #387 First Unique Character

MEDIUM PROBLEMS:

- ◆ LeetCode #560 Subarray Sum Equals K
- ◆ LeetCode #49 Group Anagrams
- ◆ LeetCode #347 Top K Frequent Elements

WHEN NOT TO USE: When space is severely constrained — bit manipulation or sorting might be better

■ 14. GREEDY BASICS

WHAT IT SOLVES	Make the locally optimal choice at each step, hoping it leads to global optimum
WHY IT EXISTS	Sometimes the best choice right now is the best choice overall. Greedy avoids exploring all possibilities — it picks the best current option and moves on.
REAL-WORLD ANALOGY	Paying with coins: to make \$0.67, always use the largest coin that fits. \$0.50 + \$0.10 + \$0.05 + \$0.01 + \$0.01. Locally optimal = globally optimal here.

INTUITION

Sort by some criterion, then greedily assign. The key: prove that greedy choice doesn't eliminate better future choices (greedy correctness).

DRY RUN / STEP-BY-STEP THINKING:

- Step 1: Identify what to 'greedily' maximize or minimize
- Step 2: Sort input by relevant criterion (deadline, ratio, etc.)
- Step 3: Iterate and take greedy choice at each step
- Step 4: Prove: does greedy choice ever make a future choice worse?

JAVA IMPLEMENTATION:

```
// Activity Selection (classic greedy)
// Given activities with start/end, select max non-overlapping
public static int maxActivities(int[] start, int[] end) {
    // Sort by end time (greedy: finish early, free up time)
    // Using index sort for demo
    int count = 1, lastEnd = end[0];
    for (int i = 1; i < start.length; i++) {
        if (start[i] >= lastEnd) {
            count++;
            lastEnd = end[i];
        }
    }
    return count;
}
```

Time Complexity	Space Complexity
Usually $O(n \log n)$ due to sorting, $O(n)$ for iteration	$O(1)$ to $O(n)$ depending on problem

BEGINNER MISTAKES:

- *Applying greedy when it doesn't work (greedy requires proof of correctness)*
- *Not sorting by the correct criterion*
- *Confusing greedy with dynamic programming — if greedy choices conflict, use DP*

INTERVIEW IMPORTANCE: ★★★★★ — Jump Game, Gas Station, Meeting Rooms, Coin Change (if denominations allow greedy)

EASY PROBLEMS:

- ✓ LeetCode #455 Assign Cookies
- ✓ LeetCode #860 Lemonade Change
- ✓ LeetCode #1005 Maximize Sum After K Negations

MEDIUM PROBLEMS:

- ◆ LeetCode #55 Jump Game
- ◆ LeetCode #45 Jump Game II
- ◆ LeetCode #435 Non-overlapping Intervals

WHEN NOT TO USE: When greedy choice eliminates optimal future choices — use Dynamic Programming instead

■ 15. RECURSION BASICS

WHAT IT SOLVES	Solve problems by breaking them into smaller versions of the same problem
WHY IT EXISTS	Some problems are naturally self-similar: $\text{factorial}(n) = n * \text{factorial}(n-1)$. Recursion expresses this elegantly and is the foundation of trees, graphs, and backtracking.
REAL-WORLD ANALOGY	Russian dolls (Matryoshka): open a doll, find a smaller same-shaped doll inside. The pattern repeats until you reach the smallest doll (base case).
INTUITION	Every recursive function needs: 1) Base case (when to stop), 2) Recursive call (smaller problem), 3) Return and combine. Think: what is the smallest version of this problem?

DRY RUN / STEP-BY-STEP THINKING:

- Step 1: Identify the BASE CASE: the simplest input with known answer
- Step 2: Identify the RECURSIVE CASE: how to reduce to smaller problem
- Step 3: Write base case FIRST (most common mistake: forgetting it)
- Step 4: Trust the recursion: assume the recursive call works correctly
- Step 5: Combine: use the recursive result to build current answer

JAVA IMPLEMENTATION:

```
// Factorial - n! = n * (n-1)!
public static int factorial(int n) {
    if (n == 0 || n == 1) return 1; // Base case
    return n * factorial(n - 1);    // Recursive case
}

// Fibonacci
public static int fib(int n) {
    if (n <= 1) return n;          // Base case
    return fib(n-1) + fib(n-2);    // Recursive case
}

// Print 1 to N
public static void print1toN(int n) {
    if (n == 0) return;           // Base case
    print1toN(n - 1);             // Recurse first (print smaller)
    System.out.println(n);        // Print on the way back
}
```

Time Complexity	Space Complexity
Depends on depth and work per call. Factorial: $O(n)$ time $O(n)$ stack. Fibonacci naive: $O(2^n)$.	$O(n)$ stack space for depth-n recursion

BEGINNER MISTAKES:

- Forgetting base case → *StackOverflowError*

- *Wrong base case: off-by-one ($n < 0$ vs $n == 0$ vs $n == 1$)*
- *Not trusting recursion: trying to trace every call in your head (unnecessary for deep recursion)*
- *Using recursion where iteration is clearer (adds complexity without benefit)*

INTERVIEW IMPORTANCE: ★★★★★ — Foundation of all tree problems, DFS, backtracking, divide and conquer. Non-negotiable.

EASY PROBLEMS:

- ✓ Factorial, Fibonacci, Sum 1 to N
- ✓ Power function
- ✓ Print array recursively
- ✓ Reverse string recursively

MEDIUM PROBLEMS:

- ◆ Tower of Hanoi
- ◆ Check palindrome recursively
- ◆ LeetCode #344 Reverse String recursive
- ◆ LeetCode #509 Fibonacci Number

WHEN NOT TO USE: When iteration is simpler and stack depth could cause overflow — use iterative approach

■ 16. BACKTRACKING INTRODUCTION

WHAT IT SOLVES	Explore all possible solutions by trying options, and 'backtrack' when a path fails
WHY IT EXISTS	When brute force explores all combinations but we need to prune invalid paths early. Backtracking is 'smart brute force' — it abandons paths that can't lead to solution.
REAL-WORLD ANALOGY	A maze: at each junction, try all directions. Hit a wall? Go back to last junction and try the next direction. Continue until exit found.
INTUITION	Try a choice → if valid, recurse deeper → if invalid or complete, undo the choice (backtrack) → try next option.

DRY RUN / STEP-BY-STEP THINKING:

- Step 1: Define: what is a 'choice' and what is 'valid'?
- Step 2: Make a choice (add to current path)
- Step 3: If solution complete → record it
- Step 4: Else → recurse deeper
- Step 5: Undo the choice (backtrack) ← this is the key step
- Step 6: Try the next choice

JAVA IMPLEMENTATION:

```
// Generate all subsets of [1,2,3]
public static void subsets(int[] nums, int start,
                          List current, List<List> result) {
    result.add(new ArrayList<>(current)); // Every state is a valid subset
    for (int i = start; i < nums.length; i++) {
```

```

current.add(nums[i]);           // Make choice
subsets(nums, i + 1, current, result); // Recurse
current.remove(current.size() - 1); // Backtrack (undo choice)
}
}

```

Time Complexity	Space Complexity
Exponential in worst case (2^n for subsets, $n!$ for permutations). Pruning reduces practical runtime.	$O(n)$ recursion stack depth

BEGINNER MISTAKES:

- *Forgetting to undo the choice (backtrack step)*
- *Not passing a copy of the list when recording results (result.add(current) adds reference!)*
- *Not identifying when to prune — add constraints early to avoid unnecessary exploration*

INTERVIEW IMPORTANCE: ★★★★★ — Subsets, permutations, N-Queens, Sudoku solver. All combinatorial problems use this.

EASY PROBLEMS:

- ✓ LeetCode #78 Subsets
- ✓ LeetCode #46 Permutations
- ✓ Generate all binary strings of length n

MEDIUM PROBLEMS:

- ◆ LeetCode #90 Subsets II
- ◆ LeetCode #39 Combination Sum
- ◆ LeetCode #51 N-Queens

WHEN NOT TO USE: When order doesn't matter and only one solution needed — use greedy or DP if pattern fits

■ **17. BIT MANIPULATION BASICS**

WHAT IT SOLVES	Use binary representations and bitwise operators for ultra-efficient solutions
WHY IT EXISTS	Computers store everything in binary. Bit operations run at hardware speed — they're the fastest possible operations. Many integer problems have elegant $O(1)$ bit solutions.
REAL-WORLD ANALOGY	Light switches as bits: 0 = off, 1 = on. AND = both must be on. OR = either on. XOR = exactly one on. Left shift = multiply by 2. Right shift = divide by 2.
INTUITION	Learn 6 operators: AND (&), OR (), XOR (^), NOT (~), Left Shift (<<), Right Shift (>>). Then memorize key tricks: $n \& (n-1)$ removes last set bit, $n \& 1$ checks odd/even.

DRY RUN / STEP-BY-STEP THINKING:

- Step 1: $n \& 1 \rightarrow$ check if n is odd (last bit)
- Step 2: $n \gg 1 \rightarrow$ divide by 2 (right shift)
- Step 3: $n \ll 1 \rightarrow$ multiply by 2 (left shift)
- Step 4: $n \& (n-1) \rightarrow$ clear lowest set bit (count set bits)
- Step 5: $n \wedge n = 0$, $n \wedge 0 = n \rightarrow$ XOR tricks for duplicates
- Step 6: $1 \ll k \rightarrow$ create mask with k th bit set

JAVA IMPLEMENTATION:

```
// Count set bits (Brian Kernighan's algorithm)
public static int countBits(int n) {
    int count = 0;
    while (n != 0) {
        n = n & (n - 1); // Clear lowest set bit
        count++;
    }
    return count;
}

// Check power of 2
public static boolean isPowerOf2(int n) {
    return n > 0 && (n & (n-1)) == 0;
}

// Find single non-duplicate (XOR of all elements)
public static int singleNumber(int[] nums) {
    int result = 0;
    for (int n : nums) result ^= n;
    return result;
}
```

Time Complexity	Space Complexity
$O(1)$ or $O(\log n)$ for most bit operations	$O(1)$ — no extra space

BEGINNER MISTAKES:

- *Operator precedence: $(n \& 1) == 0$ needs parentheses, $n \& 1 == 0$ is wrong!*
- *Using bit tricks on negative numbers without understanding two's complement*
- *Overcomplicating: only use when bit approach is clearly simpler*

INTERVIEW IMPORTANCE: ★★★★★ — Single Number (XOR), Power of 2, Count Bits are interview favorites.

EASY PROBLEMS:

- ✓ LeetCode #136 Single Number
- ✓ LeetCode #191 Number of 1 Bits
- ✓ LeetCode #231 Power of Two

MEDIUM PROBLEMS:

- ◆ LeetCode #137 Single Number II
- ◆ LeetCode #260 Single Number III
- ◆ LeetCode #338 Counting Bits

WHEN NOT TO USE: When problem involves floating point or doesn't naturally map to binary — avoid bit tricks

■ ■ PHASE 3 — DSA FOUNDATION | WEEK 5–6

You have learned algorithms and patterns. Now you'll apply them to DATA STRUCTURES — the containers that organize data. Each DSA topic comes with a complete Problem Completion Roadmap.

■ 1. ARRAYS DSA

CONCEPT	A contiguous block of memory storing elements of the same type. Elements are accessed via index in $O(1)$. Foundation of all data structures.
WHY IT EXISTS	Random access ($O(1)$) makes arrays essential for algorithms that need direct element access. Understanding arrays deeply is understanding how memory works.
ANALOGY	A row of numbered lockers in school: locker 0, locker 1, locker 2... Access any locker directly by its number. But inserting in the middle requires shifting all lockers.
VISUALIZATION	<code>arr[]</code> → <code>[10][20][30][40][50]</code> → index 0,1,2,3,4. Memory addresses are contiguous. <code>arr[2] = base_address + 2*element_size</code> .

JAVA IMPLEMENTATION:

```
// Declaration and initialization
int[] arr = new int[5];           // [0,0,0,0,0]
int[] arr2 = {1, 2, 3, 4, 5};    // Direct initialization
int[][] matrix = new int[3][4];  // 3 rows, 4 cols

// Common operations
int len = arr.length;            // No () - it's a property
Arrays.sort(arr);                //  $O(n \log n)$ 
Arrays.fill(arr, 0);            // Fill with 0
int[] copy = Arrays.copyOf(arr, arr.length);
System.out.println(Arrays.toString(arr)); // Print array
```

OPERATION COMPLEXITIES:

Access $O(1)$ | Search $O(n)$ | Insert at end $O(1)$ amortized | Insert at index $O(n)$ | Delete $O(n)$

BEGINNER MISTAKES:

- Using `arr.length()` with `()` — arrays use `.length` without parentheses
- `ArrayIndexOutOfBoundsException`: accessing index `n` when valid range is 0 to `n-1`
- Modifying array while iterating with for-each loop
- Forgetting arrays are passed by reference — changes inside method affect original

■ PROBLEM COMPLETION ROADMAP

EASY — Solve First	MEDIUM — Solve Next	HARD — Attempt Later
LC #1 Two Sum • LC #26 Remove Duplicates • LC #283 Move Zeros • LC #905 Sort by Parity • LC #1431 Kids with Candies	LC #11 Container With Most Water • LC #15 3Sum • LC #238 Product Except Self • LC #152 Maximum Product Subarray	LC #42 Trapping Rain Water • LC #84 Largest Rectangle in Histogram • LC #41 First Missing Positive
Minimum Problems	Days to Spend	Ready When
Minimum 15 problems (8 Easy + 5 Medium + 2 Hard)	4–5 days	Can solve any Easy in < 20 min and Medium in < 40 min without hints

■ 2. STRINGS DSA

CONCEPT	A sequence of characters. In Java, String is immutable — every modification creates a new String. Use StringBuilder for efficient string building.
WHY IT EXISTS	Text processing is fundamental. Understanding string immutability, character arrays, and string methods is essential for interview problems.
ANALOGY	A string is like a bead necklace: each bead is a character. You can't change a bead (immutable), but you can make a new necklace with modifications.
VISUALIZATION	'hello' → ['h']['e']['l']['l']['o'] → index 0,1,2,3,4. charAt(2) = 'l'. substring(1,3) = 'el' (end exclusive).

JAVA IMPLEMENTATION:

```
// Key String methods
String s = "Hello world";
int len = s.length();           // 11
char c = s.charAt(0);           // 'H'
String sub = s.substring(0, 5); // "Hello" (end exclusive)
String lower = s.toLowerCase(); // "hello world"
boolean has = s.contains("World"); // true
String[] parts = s.split(" "); // ["Hello", "World"]
String rev = new StringBuilder(s).reverse().toString();

// IMPORTANT: Use StringBuilder for building strings!
StringBuilder sb = new StringBuilder();
sb.append("Hello").append(" ").append("World");
String result = sb.toString();
```

OPERATION COMPLEXITIES:

charAt O(1) | substring O(n) | contains O(n*m) | StringBuilder append O(1) amortized

BEGINNER MISTAKES:

- Using + in loops to concatenate strings (O(n²)) — use StringBuilder instead
- Comparing strings with == instead of .equals()

■ *Off-by-one in substring: `s.substring(0,3)` gives characters 0,1,2 — NOT including index 3*

■ *Forgetting strings are immutable: `s.toUpperCase()` doesn't modify `s`, returns new string*

■ PROBLEM COMPLETION ROADMAP

EASY — Solve First	MEDIUM — Solve Next	HARD — Attempt Later
• LC #344 Reverse String • LC #387 First Unique Character • LC #242 Valid Anagram • LC #125 Valid Palindrome • LC #14 Longest Common Prefix	• LC #3 Longest Substring Without Repeating • LC #5 Longest Palindromic Substring • LC #49 Group Anagrams • LC #151 Reverse Words in String	• LC #10 Regular Expression Matching • LC #72 Edit Distance • LC #76 Minimum Window Substring
Minimum Problems	Days to Spend	Ready When
Minimum 12 problems (6 Easy + 4 Medium + 2 Hard)	3–4 days	Can implement palindrome, anagram, reverse word problems without reference

■ 3. RECURSION DEEP DIVE

CONCEPT	A function calling itself with a smaller input until reaching the base case. The call stack stores each function call's state until base case returns.
WHY IT EXISTS	Trees, graphs, divide & conquer, backtracking — all are fundamentally recursive. You cannot master advanced DSA without deep recursion understanding.
ANALOGY	A mirror facing another mirror: each reflection contains a smaller version of itself. The smallest reflection is the base case.
VISUALIZATION	$\text{factorial}(3) \rightarrow 3 * \text{factorial}(2) \rightarrow 3 * 2 * \text{factorial}(1) \rightarrow 3 * 2 * 1 * \text{factorial}(0) \rightarrow 3 * 2 * 1 * 1 = 6$. Call stack unwinds upward.

JAVA IMPLEMENTATION:

```
// Recursion template
public static ReturnType solve(params) {
    // 1. BASE CASE (most important!)
    if (base_condition) return base_value;

    // 2. RECURSIVE CASE (reduce problem)
    return combine(current_value, solve(smaller_params));
}

// Example: Sum of array elements
public static int arraySum(int[] arr, int idx) {
    if (idx == arr.length) return 0; // Base case
    return arr[idx] + arraySum(arr, idx + 1);
}

// Example: Binary Search recursive
public static int bSearch(int[] arr, int l, int r, int t) {
    if (l > r) return -1;
    int mid = l + (r - l) / 2;
    if (arr[mid] == t) return mid;
    if (arr[mid] < t) return bSearch(arr, mid+1, r, t);
    return bSearch(arr, l, mid-1, t);
}
```

OPERATION COMPLEXITIES:

Depends on depth. Stack space $O(\text{depth})$. Time depends on work per call and branching factor.

BEGINNER MISTAKES:

- *Missing base case $\rightarrow$ StackOverflowError*
- *Wrong base case condition (off by one)*
- *Not trusting recursion — don't trace every level, trust the function works for $n-1$*
- *Modifying shared state without backtracking*

■ PROBLEM COMPLETION ROADMAP

EASY — Solve First	MEDIUM — Solve Next	HARD — Attempt Later
- LC #509 Fibonacci - LC #206 Reverse Linked List (recursive) - LC #21 Merge Two Sorted Lists - Print 1 to N recursively - Sum of digits recursively	- LC #78 Subsets - LC #46 Permutations - LC #779 Kth Symbol in Grammar - Tower of Hanoi	- LC #51 N-Queens - LC #37 Sudoku Solver - LC #301 Remove Invalid Parentheses
Minimum Problems	Days to Spend	Ready When
Minimum 10 problems of varying depth	4 days	Can write recursive solution without seeing base case fail, and can convert to iterative

■ 4. LINKED LIST

CONCEPT	A linear data structure where each element (node) contains data and a pointer to the next node. Not contiguous in memory — each node can be anywhere.
WHY IT EXISTS	Unlike arrays, insertion/deletion at the beginning or middle is $O(1)$ once you have the pointer. Used in stacks, queues, hash chaining, memory allocation.
ANALOGY	A treasure hunt: each clue (node) has data (the clue) and a pointer (location of next clue). To get to clue 5, you must follow all clues from 1 to 5 — no random access.
VISUALIZATION	head → [10 next] → [20 next] → [30 next] → [40 null]. Each node: int data + ListNode next.

JAVA IMPLEMENTATION:

```
// Node definition
class ListNode {
    int val;
    ListNode next;
    ListNode(int val) { this.val = val; }
}

// Common operations
public ListNode reverse(ListNode head) {
    ListNode prev = null, curr = head;
    while (curr != null) {
        ListNode next = curr.next;
        curr.next = prev;
        prev = curr;
        curr = next;
    }
    return prev;
}

public int length(ListNode head) {
    int count = 0;
    while (head != null) { head = head.next; count++; }
}
```

```
return count;
}
```

OPERATION COMPLEXITIES:

Access $O(n)$ | Search $O(n)$ | Insert at head $O(1)$ | Insert at tail $O(n)$ or $O(1)$ with tail pointer | Delete $O(1)$ with pointer

BEGINNER MISTAKES:

- *NullPointerException: accessing node.next when node is null — always check `!= null`*
- *Losing the head pointer — save head before modifying*
- *Off-by-one in traversal: stopping at last node vs stopping after last node*
- *Not handling empty list or single-node list cases*

PROBLEM COMPLETION ROADMAP

EASY — Solve First	MEDIUM — Solve Next	HARD — Attempt Later
• LC #206 Reverse Linked List • LC #876 Middle of Linked List • LC #21 Merge Two Sorted Lists • LC #83 Remove Duplicates • LC #141 Linked List Cycle	• LC #19 Remove Nth from End • LC #142 Linked List Cycle II • LC #160 Intersection of Lists • LC #24 Swap Nodes in Pairs	• LC #25 Reverse Nodes in k-Group • LC #148 Sort List (Merge Sort) • LC #23 Merge K Sorted Lists
Minimum Problems	Days to Spend	Ready When
Minimum 12 problems (5 Easy + 5 Medium + 2 Hard)	4–5 days	Can code reverse, detect cycle, find middle without reference in < 15 minutes

■ 5. STACK

CONCEPT	A LIFO (Last In, First Out) data structure. The last element pushed is the first to be popped. Think of a stack of plates.
WHY IT EXISTS	Essential for: function call management (call stack), expression evaluation, undo functionality, DFS traversal, balanced parentheses checking.
ANALOGY	A stack of plates: you can only add (push) or remove (pop) from the top. The plate you placed last is the first you take back.
VISUALIZATION	Push 1 → [1]. Push 2 → [1,2]. Push 3 → [1,2,3]. Pop → returns 3, stack is [1,2]. Peek → returns 2 without removing.

JAVA IMPLEMENTATION:

```
// Java Stack using Deque (recommended over Stack class)
Deque stack = new ArrayDeque<>();
stack.push(10);      // Push to top
stack.push(20);
int top = stack.peek(); // See top: 20 (no remove)
int val = stack.pop();  // Remove top: 20
boolean empty = stack.isEmpty(); // Check if empty
int size = stack.size();

// Classic problem: Valid Parentheses
public boolean isValid(String s) {
    Deque stack = new ArrayDeque<>();
    for (char c : s.toCharArray()) {
        if (c=='(' || c=='{' || c=='[') stack.push(c);
        else {
            if (stack.isEmpty()) return false;
            char top = stack.pop();
            if (c==')' && top!='(') return false;
            if (c=='}' && top!='{') return false;
            if (c==']' && top!='[') return false;
        }
    }
    return stack.isEmpty();
}
```

OPERATION COMPLEXITIES:

Push $O(1)$ | Pop $O(1)$ | Peek $O(1)$ | Search $O(n)$

BEGINNER MISTAKES:

- Using Java's Stack class (legacy, use ArrayDeque instead)
- Forgetting to check isEmpty() before pop() → EmptyStackException
- Confusing LIFO with FIFO — stack is LIFO, queue is FIFO

■ PROBLEM COMPLETION ROADMAP

EASY — Solve First	MEDIUM — Solve Next	HARD — Attempt Later
• LC #20 Valid Parentheses • LC #155 Min Stack • LC #232 Implement Queue using Stacks • Reverse a string using stack	• LC #739 Daily Temperatures • LC #496 Next Greater Element I • LC #150 Evaluate RPN • LC #84 Largest Rectangle	• LC #85 Maximal Rectangle • LC #42 Trapping Rain Water (stack approach) • LC #32 Longest Valid Parentheses
Minimum Problems	Days to Spend	Ready When
Minimum 10 problems (4 Easy + 4 Medium + 2 Hard)	3 days	Can solve Next Greater Element and Valid Parentheses without hints

■ 6. QUEUE

CONCEPT	A FIFO (First In, First Out) data structure. The first element enqueued is the first to be dequeued. Think of a real queue/line.
WHY IT EXISTS	Essential for: BFS traversal (most important use case), task scheduling, printer queue, level-order tree traversal.
ANALOGY	A queue at a bus stop: first person in line boards first. New people join the back. Nobody cuts in line.
VISUALIZATION	Enqueue 1 → [1]. Enqueue 2 → [1,2]. Enqueue 3 → [1,2,3]. Dequeue → removes 1. Queue is [2,3]. Front = 2.

JAVA IMPLEMENTATION:

```
// Java Queue using ArrayDeque
Queue queue = new ArrayDeque<>();
queue.offer(10);      // Enqueue (add to back)
queue.offer(20);
int front = queue.peek(); // See front: 10
int val = queue.poll();  // Remove front: 10
boolean empty = queue.isEmpty();

// BFS template using Queue
public void bfs(int start, List<Integer> graph) {
    Queue queue = new ArrayDeque<>();
    boolean[] visited = new boolean[graph.size()];
    queue.offer(start);
    visited[start] = true;
    while (!queue.isEmpty()) {
        int node = queue.poll();
        System.out.println("Visited: " + node);
        for (int neighbor : graph.get(node)) {
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                queue.offer(neighbor);
            }
        }
    }
}
```

```
}
}
```

OPERATION COMPLEXITIES:

Enqueue $O(1)$ | Dequeue $O(1)$ | Peek $O(1)$ | Search $O(n)$

BEGINNER MISTAKES:

- Using Queue for DFS (use Stack for DFS, Queue for BFS)
- Calling `remove()` on empty queue throws exception — use `poll()` which returns null
- Not marking nodes as visited in BFS → infinite loop on cyclic graphs

PROBLEM COMPLETION ROADMAP

EASY — Solve First	MEDIUM — Solve Next	HARD — Attempt Later
• LC #232 Implement Stack using Queues • LC #102 Binary Tree Level Order Traversal • LC #225 Implement Stack using Queues • Print nodes at each level	• LC #286 Walls and Gates • LC #994 Rotting Oranges • LC #127 Word Ladder • LC #542 01 Matrix	• LC #407 Trapping Rain Water II • LC #317 Shortest Distance from All Buildings
Minimum Problems	Days to Spend	Ready When
Minimum 8 problems — focus on BFS problems as they need Queue	3 days	Can write BFS template from memory and solve level-order traversal

■ 7. HASHMAP & 8. HASHSET

CONCEPT	HashMap: Key-value pairs with $O(1)$ average get/put. HashSet: Unique elements only, $O(1)$ add/contains. Both use hashing internally.
WHY IT EXISTS	Linear search in arrays is $O(n)$. HashMap provides $O(1)$ lookup for any key. This transforms many $O(n^2)$ problems to $O(n)$.
ANALOGY	HashMap = a dictionary: look up any word (key) instantly to find its definition (value). HashSet = a VIP list: instantly check if someone is on it.
VISUALIZATION	put(key, value) → hashes key → stores at hash location. get(key) → hashes key → retrieves value. $O(1)$ average because hash function computes location directly.

JAVA IMPLEMENTATION:

```
// HashMap
Map map = new HashMap<>();
map.put("apple", 5);
map.put("banana", 3);
int val = map.get("apple");           // 5
boolean has = map.containsKey("cherry"); // false
map.getDefault("cherry", 0);         // 0 (safe default)
map.put("apple", map.getDefault("apple", 0) + 1); // increment

// Iterate
for (Map.Entry entry : map.entrySet()) {
    System.out.println(entry.getKey() + ": " + entry.getValue());
}

// HashSet
Set set = new HashSet<>();
set.add(1); set.add(2); set.add(1); // {1, 2} - no duplicates
boolean has2 = set.contains(2);     // true, O(1)
set.remove(1);
```

OPERATION COMPLEXITIES:

get $O(1)$ avg | put $O(1)$ avg | containsKey $O(1)$ avg | Iteration $O(n)$ | Worst case $O(n)$ with collisions

BEGINNER MISTAKES:

- Using == for key comparison (HashMap uses .equals() internally — important for custom objects)
- Assuming HashMap maintains insertion order (use LinkedHashMap for that)
- ConcurrentModificationException: modifying map while iterating — use iterator.remove()

■ PROBLEM COMPLETION ROADMAP

EASY — Solve First**MEDIUM — Solve Next****HARD — Attempt Later**

• LC #1 Two Sum • LC #217 Contains Duplicate • LC #242 Valid Anagram • LC #383 Ransom Note • Count frequency of elements	• LC #49 Group Anagrams • LC #347 Top K Frequent Elements • LC #560 Subarray Sum Equals K • LC #525 Contiguous Array	• LC #76 Minimum Window Substring • LC #30 Substring with Concatenation of All Words
Minimum Problems	Days to Spend	Ready When
Minimum 12 problems — HashMap is in 40% of all interview problems	4 days	Can choose between HashMap/HashSet/Array for frequency counting and explain why

■ 9. TREES & 10. BINARY SEARCH TREES

CONCEPT	Tree: hierarchical data structure with root, nodes, and edges. BST: Binary Tree where left subtree < root < right subtree. Enables $O(\log n)$ search.
WHY IT EXISTS	Trees model hierarchical relationships (file system, HTML DOM, organization chart). BST enables $O(\log n)$ search, insert, delete vs $O(n)$ for arrays.
ANALOGY	A family tree: one root (grandparent), branching down to children and grandchildren. BST is like a sorted family tree where left branch always has smaller values.
VISUALIZATION	Root=10. Left subtree: 5 (left:3, right:7). Right subtree: 15 (left:12, right:20). BST property: $3 < 5 < 7 < 10 < 12 < 15 < 20$.

JAVA IMPLEMENTATION:

```
class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode(int val) { this.val = val; }
}

// Inorder traversal (Left-Root-Right) = sorted order for BST
public void inorder(TreeNode root) {
    if (root == null) return;
    inorder(root.left);
    System.out.print(root.val + " ");
    inorder(root.right);
}

// BST Search
public TreeNode search(TreeNode root, int target) {
    if (root == null || root.val == target) return root;
    if (target < root.val) return search(root.left, target);
    return search(root.right, target);
}

// Tree Height
public int height(TreeNode root) {
```

```

if (root == null) return 0;
return 1 + Math.max(height(root.left), height(root.right));
}

```

OPERATION COMPLEXITIES:

BST: Search/Insert/Delete $O(\log n)$ avg, $O(n)$ worst (skewed). Traversals $O(n)$. Height balanced trees (AVL, Red-Black) guarantee $O(\log n)$ always.

BEGINNER MISTAKES:

- *Forgetting null checks before accessing root.left or root.right*
- *Confusing inorder/preorder/postorder (practice drawing all three)*
- *BST delete is the hardest operation — 3 cases: leaf, one child, two children (replace with inorder successor)*

■ PROBLEM COMPLETION ROADMAP

EASY — Solve First	MEDIUM — Solve Next	HARD — Attempt Later
• LC #104 Maximum Depth of Binary Tree • LC #226 Invert Binary Tree • LC #112 Path Sum • LC #700 Search in BST • LC #144 Binary Tree Preorder	• LC #102 Level Order Traversal • LC #236 Lowest Common Ancestor • LC #105 Construct from Preorder+Inorder • LC #98 Validate BST	• LC #124 Binary Tree Maximum Path Sum • LC #297 Serialize and Deserialize Binary Tree • LC #99 Recover BST
Minimum Problems	Days to Spend	Ready When
Minimum 15 problems — trees appear in 30% of all interviews	5–6 days	Can write all 3 traversals from memory and solve level-order, height, LCA without hints

■ 11. HEAP BASICS

CONCEPT	A complete binary tree satisfying the heap property. Max-Heap: parent $\geq$ children. Min-Heap: parent $\leq$ children. Implemented as array. Priority Queue in Java.
WHY IT EXISTS	Finding the maximum or minimum repeatedly is slow with arrays ($O(n)$ each time). Heap gives $O(\log n)$ insert and $O(\log n)$ extract-max/min. Essential for priority operations.
ANALOGY	A hospital emergency room: patients are prioritized by severity (not arrival time). The most critical patient is always treated first. This is a max-heap by severity.
VISUALIZATION	Max-Heap: [50,30,40,10,20,35,15]. Root=50 is always max. Array representation: parent of i is at $(i-1)/2$. Children of i are at $2i+1$ and $2i+2$.

JAVA IMPLEMENTATION:

```
// Java PriorityQueue (Min-Heap by default)
PriorityQueue minHeap = new PriorityQueue<>();
minHeap.offer(5); minHeap.offer(1); minHeap.offer(3);
int min = minHeap.peek(); // 1 (minimum, no remove)
int val = minHeap.poll(); // 1 (remove minimum)

// Max-Heap using comparator
PriorityQueue maxHeap = new PriorityQueue<>(
    Collections.reverseOrder()
);
maxHeap.offer(5); maxHeap.offer(1); maxHeap.offer(3);
int max = maxHeap.poll(); // 5 (maximum)

// Kth Largest Element using min-heap of size k
public int findKthLargest(int[] nums, int k) {
    PriorityQueue heap = new PriorityQueue<>();
    for (int num : nums) {
        heap.offer(num);
        if (heap.size() > k) heap.poll(); // Remove smallest
    }
    return heap.peek(); // Top of heap = kth largest
}
```

OPERATION COMPLEXITIES:

Insert $O(\log n)$ | Extract-Min/Max $O(\log n)$ | Peek $O(1)$ | Build heap from array $O(n)$

BEGINNER MISTAKES:

- *Confusing Min-Heap and Max-Heap direction*
- *PriorityQueue in Java is MIN-heap — for max use reverseOrder()*
- *Heap does NOT sort the entire array — only guarantees root is min/max*

■ PROBLEM COMPLETION ROADMAP

EASY — Solve First

MEDIUM — Solve Next

HARD — Attempt Later

• LC #215 Kth Largest Element • LC #703 Kth Largest in Stream • LC #1046 Last Stone Weight • Find K closest points to origin	• LC #347 Top K Frequent Elements • LC #373 Find K Pairs with Smallest Sums • LC #378 Kth Smallest in Sorted Matrix	• LC #295 Find Median from Data Stream • LC #23 Merge K Sorted Lists • LC #407 Trapping Rain Water II
Minimum Problems	Days to Spend	Ready When
Minimum 8 problems — focus on 'top K' pattern	3 days	Can choose between min/max heap appropriately and solve Kth largest in < 20 min

■ 12. GRAPH BASICS

CONCEPT	A collection of nodes (vertices) connected by edges. Can be directed/undirected, weighted/unweighted. Most flexible data structure.
WHY IT EXISTS	Graphs model: social networks (friends), maps (cities+roads), dependency graphs, internet. BFS finds shortest path in unweighted graphs. DFS finds connectivity.
ANALOGY	A city map: cities are vertices, roads are edges. Facebook friends is an undirected graph. Twitter follows is a directed graph. Google Maps uses weighted graphs.
VISUALIZATION	Adjacency List: graph.get(0) = [1,2] (vertex 0 connects to 1 and 2). More memory-efficient than adjacency matrix for sparse graphs.

JAVA IMPLEMENTATION:

```
// Graph representation using adjacency list
List<List> graph = new ArrayList<>();
for (int i = 0; i < n; i++) graph.add(new ArrayList<>());
graph.get(0).add(1); graph.get(1).add(0); // Undirected edge 0-1

// DFS - Depth First Search
public void dfs(int node, boolean[] visited, List<List> graph) {
    visited[node] = true;
    System.out.print(node + " ");
    for (int neighbor : graph.get(node)) {
        if (!visited[neighbor]) dfs(neighbor, visited, graph);
    }
}

// BFS - Breadth First Search (shortest path unweighted)
public int[] bfsShortestPath(int start, List<List> graph) {
    int n = graph.size();
    int[] dist = new int[n];
    Arrays.fill(dist, -1);
    Queue queue = new ArrayDeque<>();
    queue.offer(start); dist[start] = 0;
    while (!queue.isEmpty()) {
        int node = queue.poll();
```

```

    for (int neighbor : graph.get(node)) {
        if (dist[neighbor] == -1) {
            dist[neighbor] = dist[node] + 1;
            queue.offer(neighbor);
        }
    }
}
return dist;
}

```

OPERATION COMPLEXITIES:

BFS/DFS: $O(V+E)$ where V =vertices, E =edges. Dijkstra: $O((V+E) \log V)$ with heap.

BEGINNER MISTAKES:

- *Not marking nodes visited → infinite loop in cyclic graphs*
- *Confusing BFS (level-by-level, shortest path) with DFS (depth-first, connectivity)*
- *Using adjacency matrix for sparse graphs wastes memory $O(V^2)$*

PROBLEM COMPLETION ROADMAP

EASY — Solve First	MEDIUM — Solve Next	HARD — Attempt Later
• LC #200 Number of Islands • LC #733 Flood Fill • LC #463 Island Perimeter • LC #997 Find the Town Judge	• LC #207 Course Schedule (cycle detection) • LC #994 Rotting Oranges • LC #130 Surrounded Regions • LC #127 Word Ladder	• LC #332 Reconstruct Itinerary • LC #787 Cheapest Flights within K Stops • LC #1192 Critical Connections
Minimum Problems	Days to Spend	Ready When
Minimum 12 problems — graphs are 15–20% of FAANG interviews	5–6 days	Can implement both BFS and DFS from memory and solve Number of Islands without hints

■ PHASE 4 — INTERVIEW PREPARATION & ADVANCED THINKING | WEEK 7–8

How Top Programmers Think

The difference between a programmer who gets stuck and one who solves any problem isn't talent — it's a systematic approach. Here is exactly how top engineers approach unseen problems.

→ Pattern Recognition	They've solved 500+ problems, so they recognize 'this is a sliding window' in 30 seconds. Build this library through volume.
→ Brute Force First	They always state the brute force solution first, even in interviews. It shows understanding. Then optimize.
→ Complexity Awareness	They always ask: what's my current approach's complexity? Can I do better? They know the theoretical lower bound.
→ Communication	In interviews, they think aloud. They don't code in silence. They explain their reasoning at every step.
→ Edge Case Obsession	Empty array? Single element? All same elements? Negative numbers? Integer overflow? They think of edge cases before coding.
→ Mistake Analysis	After every wrong solution, they understand WHY it was wrong — not just accept it. They build mental models.

Daily 3–5 Hour Study Timetable

Time Block	Activity	Duration	Platform
7:00 – 7:30 AM	Review yesterday's problems (no code, just logic)	30 min	Notes/Paper
7:30 – 8:00 AM	Learn new concept / algorithm from roadmap	30 min	Notes/Code
8:00 – 9:30 AM	Solve 2–3 problems on current topic	90 min	LeetCode
BREAK	Rest, walk, eat. No screens.	30 min	—
10:00 – 11:30 AM	Solve 1 medium problem (full problem-solving process)	90 min	LeetCode
11:30 – 12:00 PM	Analyze mistakes, write what you learned	30 min	Journal

EVENING (Optional)	Watch 1 algorithm explanation video, read editorial	30 min	YouTube/LC
--------------------	---	--------	------------

Weekly Revision System

SPACED REPETITION is the scientifically proven method to retain code patterns. Solve a problem → revisit after 3 days → revisit after 7 days → revisit after 14 days.

Day	Activity	Goal
Day 1	Solve new problems with full thought process	Learn the pattern
Day 3	Re-solve Day 1 problems from scratch (no looking back)	Check if truly understood
Day 7	Re-solve Day 1 problems again + solve 1 new problem of same type	Solidify with variety
Day 14	Mixed revision: 1 problem from each topic covered so far	Build cross-topic intuition
Weekly Mock	Take a 2-hour timed LeetCode contest or mock interview	Simulate real pressure

LeetCode Roadmap — Exact Order to Follow

Do NOT random-solve. Follow this structured LeetCode path. Solve by pattern, not by topic list. Aim for 150–200 quality problems over 2 months rather than rushing 500.

Week	Focus Area	Target Problems	Key LeetCode Problems
Week 1–2	Arrays, Strings, Math	30–35 Easy	#1,#9,#13,#14,#21,#26,#27,#66,#88,#121,#125,#136,#169,#217,#242,#283,#344,#349
Week 3–4	Searching, Sorting, Two Pointer, Sliding Window	25–30 Easy/Med	#11,#15,#33,#34,#35,#75,#167,#209,#238,#268,#424,#567,#643,#704
Week 5	Linked List, Stack, Queue	20–25 Easy/Med	#20,#21,#141,#142,#155,#206,#225,#232,#739,#876,#19,#24
Week 6	Trees, BST, HashMap, Heap	25–30 Med	#94,#98,#102,#104,#105,#110,#112,#226,#236,#347,#215,#295
Week 7	Graphs, Recursion, Backtracking	20–25 Med	#200,#46,#78,#207,#733,#994,#130,#39,#51,#127
Week 8	Mixed Review + Hard Problems	20 Med/Hard	#42,#84,#85,#124,#297,#76,#23,#25,#301,#332

How to Stop Tutorial Dependency

◆ The 20-Minute Rule	Spend at minimum 20 minutes genuinely stuck before looking at hints. Real problem-solving muscles only build under resistance.
◆ Look at hints, NOT solutions	LeetCode provides hints — use those. Only look at the full solution after solving or after 45+ minutes of genuine effort.

◆ Re-solve after 3 days	After seeing a solution, close everything. In 3 days, solve the same problem again from memory. This builds genuine understanding.
◆ Teach it	Explain your solution to an imaginary student or write it in your journal. If you can't explain it, you don't understand it.
◆ Solve variations	After solving a problem, ask: 'What if the array was sorted? What if there were duplicates? What if I need all solutions?'

■ BONUS — RESOURCES, HABITS & MINDSET

Best Free Coding Practice Websites

Platform	Best For	Difficulty	Java Support	URL
LeetCode	Interview prep, company-specific problems, contests	Easy → Hard	■ Excellent	leetcode.com
HackerRank	Beginners, structured learning paths, certification	Easy → Medium	■ Excellent	hackerrank.com
Codeforces	Competitive programming, contest practice	Medium → Hard	■ Good	codeforces.com
GeeksForGeeks	DSA theory, explanations, company questions	All levels	■ Excellent	geeksforgeeks.org
CodeChef	Competitive programming, Indian company contests	Easy → Hard	■ Good	codechef.com
Naukri Code360	Company-specific DSA, free mock interviews	Easy → Medium	■ Good	naukri.com/code360
InterviewBit	Structured interview prep with topics	Medium	■ Good	interviewbit.com
Visualgo	Algorithm visualization — ESSENTIAL for beginners	N/A	N/A	visualgo.net
CS50	Free Harvard computer science course	Beginner	Partial	cs50.harvard.edu

Best Books for Java, DSA & Algorithms

Book	Author	Focus	Level
Introduction to Algorithms (CLRS)	Cormen et al.	Theory + all algorithms	Advanced
Grokking Algorithms	Aditya Bhargava	Visual algorithm intro	Beginner
Cracking the Coding Interview	Gayle McDowell	FAANG interview prep	Intermediate
Java: The Complete Reference	Herbert Schildt	Complete Java language	All levels
Head First Java	Sierra & Bates	Java for visual learners	Beginner
The Algorithm Design Manual	Steven Skiena	Real-world algorithms	Intermediate
Elements of Programming Interviews in Java	Aziz, Lee, Prakash	Elite interview prep	Advanced

Best YouTube Channels for Java & DSA

Channel	Best Content	Language
NeetCode	LeetCode solutions with patterns, FAANG-focused	English
Abdul Bari	Algorithms explained with animations from scratch	English
Kunal Kushwaha	Complete Java DSA Bootcamp (FREE & excellent)	English/Hindi
Apna College	DSA in Java, structured & beginner-friendly	Hindi
CodeWithHarry	Java basics to advanced, well-structured	Hindi
Striver (take U forward)	DSA sheets, A-Z DSA course, binary search deep dives	English
William Fiset	Graph theory, tree algorithms visually explained	English
Back To Back SWE	Interview whiteboarding, logical explanations	English

Best Java IDEs

IDE	Best For	Free?
IntelliJ IDEA Community	Professional Java development, best code intelligence	■ Free Community Edition
VS Code + Java Extension Pack	Lightweight, fast startup, great for beginners	■ Completely Free
Eclipse	Industry standard, heavy but powerful	■ Free
Online: Programiz.com/java-online	No setup needed, instant coding for beginners	■ Free
Online: replit.com	Browser-based, collaborative, great for starting out	■ Free tier

Daily Habits of Strong Programmers

◆ Code Every Day — No Exceptions

Even 30 minutes on a busy day. Consistency > intensity. Your brain builds connections overnight.

◆ One Problem Deeply > Five Skimmed

Understand every line of your solution. Know 3 different approaches. Know why approach 1 is better than approach 2.

◆ Maintain a Problem Journal

Write: problem name, your approach, time complexity, what you struggled with, what pattern it belongs to. Review weekly.

◆ Embrace Being Stuck

Being stuck for 20 minutes is NORMAL and necessary. It means you're at the edge of your ability — where growth happens.

◆ No Tutorial Binge-Watching

Watching tutorials feels productive but gives zero problem-solving ability. Close YouTube. Open LeetCode.

◆ Review Solved Problems

Solved a problem last week? Try it again this week. If you can't, you didn't actually learn it.

◆ Sleep & Exercise

Code quality collapses with sleep deprivation. Exercise improves cognitive function measurably. Non-negotiable.

◆ Set Daily Goals, Not Feelings

Don't say 'I'll code when I feel like it.' Say 'I will solve 2 problems between 8-10 AM daily.' Feelings are unreliable.

Common Beginner Mistakes to Avoid

x Skipping problems because they seem hard	Hard = growth. Easy = no learning. The problems that make you feel stupid are the ones that level you up.
x Memorizing solutions	If you memorized the solution but don't understand WHY, you will fail when the problem is slightly changed. Understand, don't memorize.
x Switching languages	Comparing Java to Python is a distraction. Master one language first. The concepts are 100% transferable.
x Not testing with edge cases	An untested solution is broken code waiting to fail. Always test: empty input, single element, negative numbers, max values.
x Giving up after 1 attempt	Great programmers are not people who never fail. They're people who try 10 times with different approaches. Never give up on a problem after 1 attempt.
x Quantity over quality	Solving 300 easy problems without understanding is useless. 100 deeply understood problems beats 500 memorized ones every time.

How Many Problems to Solve Daily

Week	Daily Target	Quality Standard	Focus
Week 1–2	3–4 Easy problems	Understand every line, do dry run	Logic & Java fundamentals
Week 3–4	2–3 problems (mix Easy + Medium)	Understand time & space complexity	Algorithm patterns
Week 5–6	2–3 problems + 1 revision	Can explain approach in 2 mins	DSA application
Week 7–8	2 Medium + 1 revision + 1 hard	Solve Medium in < 30 min	Speed + accuracy
After 2 months	2–3 problems daily (maintain)	Solve hard in < 60 min	Interview readiness

FINAL MESSAGE FROM YOUR MENTOR

You now hold a complete, structured roadmap that most engineers never get. The path is clear. The resources are listed. The problems are ordered. Everything is here. But this document means nothing without ONE thing: consistency. Every great programmer you admire was once exactly where you are — confused, stuck, wondering if they'll ever 'get it'. The only difference between them and those who gave up is that they showed up the next day. And the day after that. You will get stuck. That is the process. You will feel dumb on some problems. That is the process. You will solve a hard problem one day and feel like a genius. That is also the process. Trust the process. Follow the roadmap. Solve every day. In 2 months, you will not recognize the programmer you were on Day 1. Now close this PDF and open your IDE. The code won't write itself.

■ Code. Debug. Learn. Repeat.

Java Masterclass Programming Roadmap • 2-Month Beginner to Interview-Ready